Algoritmo de *Work Stealing*

Octubre/2026



Contenido

- Preámbulo
  - Mecanismos de concurrencia con hilos en Java
- Estructura del artículo
- Introducción
- Un modelo de computación multihilos
- La propiedad de “hojas ocupadas”
- Un algoritmo de robo de trabajo (work-stealing) aleatorizado 
- Acceso atómico y el juego de reciclaje
- Análisis del algoritmo de robo de trabajo (work-stealing)
- Otros artículos para revisar
- Conclusiones y recomendaciones

Un algoritmo de robo de trabajo (work-stealing) aleatorizado

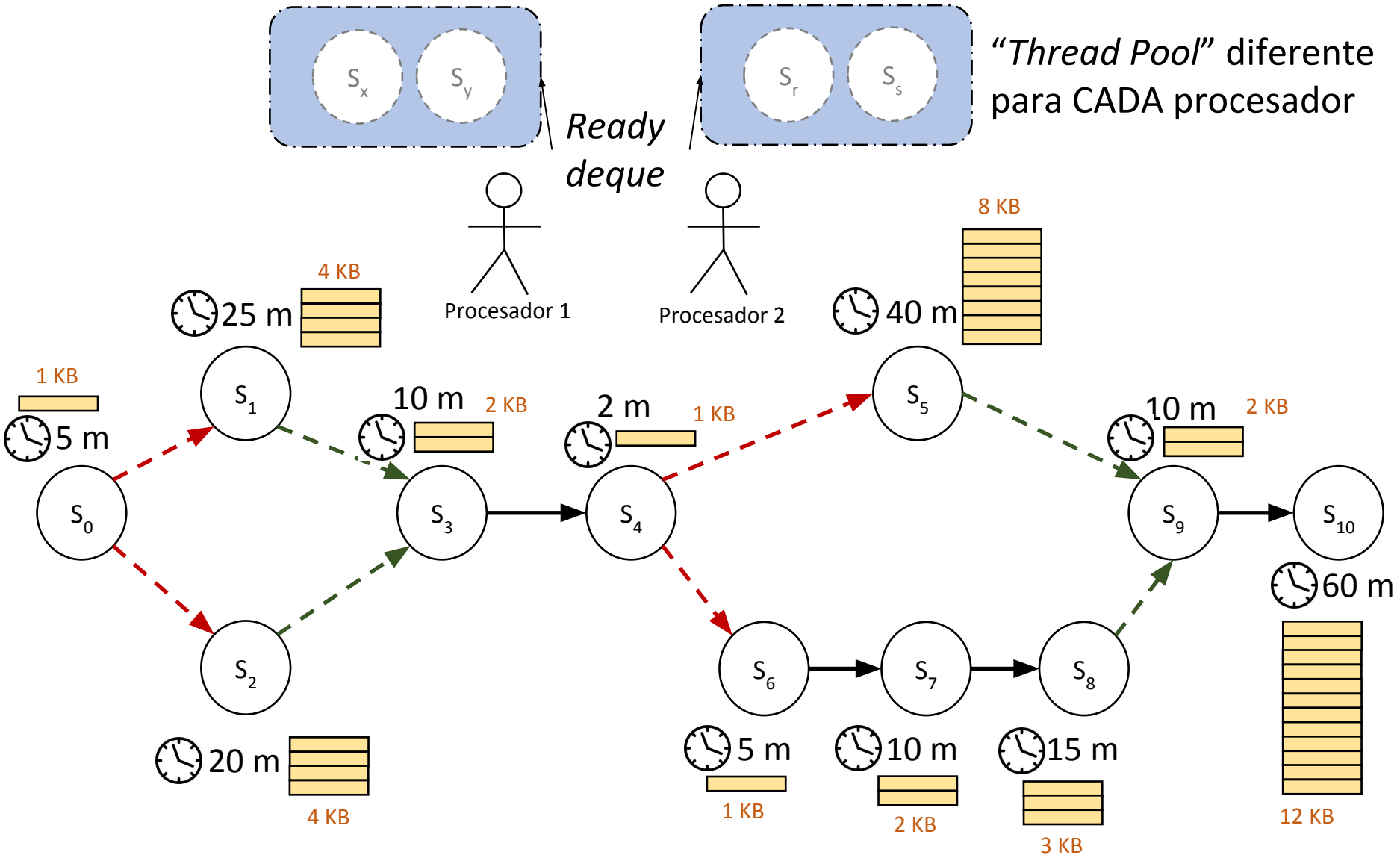
Que presenta esta sección del artículo

- Presenta un algoritmo de work-stealing (robo de trabajo) “online”, para la planificación de computaciones multihilo en computadores paralelos
- También se presenta un lema estructural importante que es utilizado al final de la sección para mostrar que para computaciones completamente estrictas, este algoritmo logra a los sumo una expansión lineal del espacio.
- Este lema reaparece en la sección 6 para mostrar que para computaciones completamente estrictas, este algoritmo logra mejora en la rapidez de computación lineal (linear speedup) y genera cantidades óptimas de comunicación.

Algoritmo “Work-stealing”

- En este algoritmo el “pool” de hilos centralizados que se utilizaba en el algoritmo de hojas ocupadas, es distribuido entre los procesadores

Algoritmo “Work-stealing”



Algoritmo “Work-stealing”

- De manera específica, cada procesador mantiene una “ready deque” (cola doblemente terminada de hilos en estado “ready”)

Recordando qué es una cola doblemente terminada (*deque*)

- Una cola doblemente terminada o *deque* (del inglés *double ended queue*) es una estructura de datos lineal que permite insertar y eliminar elementos por ambos extremos.
- Podría verse como un mecanismo que permite incorporar en una única estructura las funcionalidades de las pilas (estructuras LIFO) y las colas (estructuras FIFO)

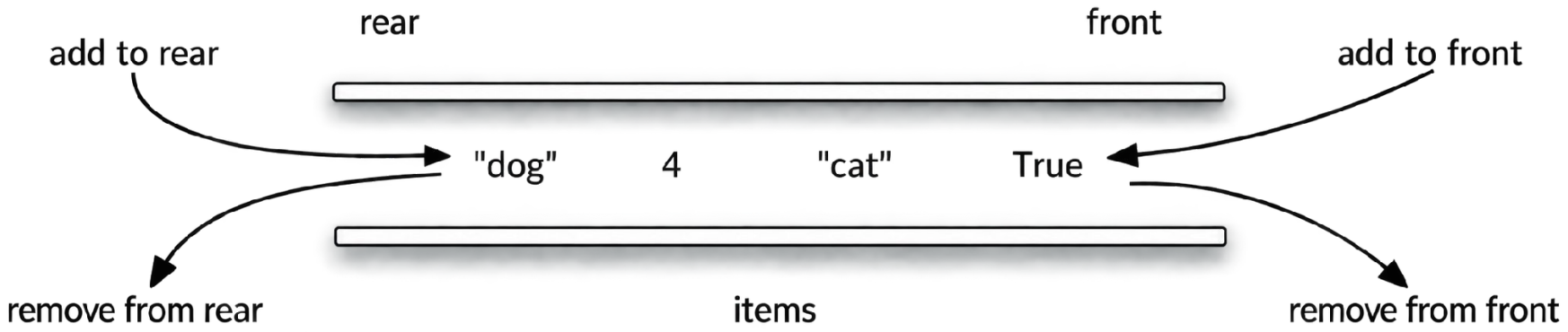


Figure 1: A Deque of Python Data Objects

https://runestone.academy/runestone/books/published/pythonds/BasicDS/WhatIsaDeque.html

Operaciones en una *deque*

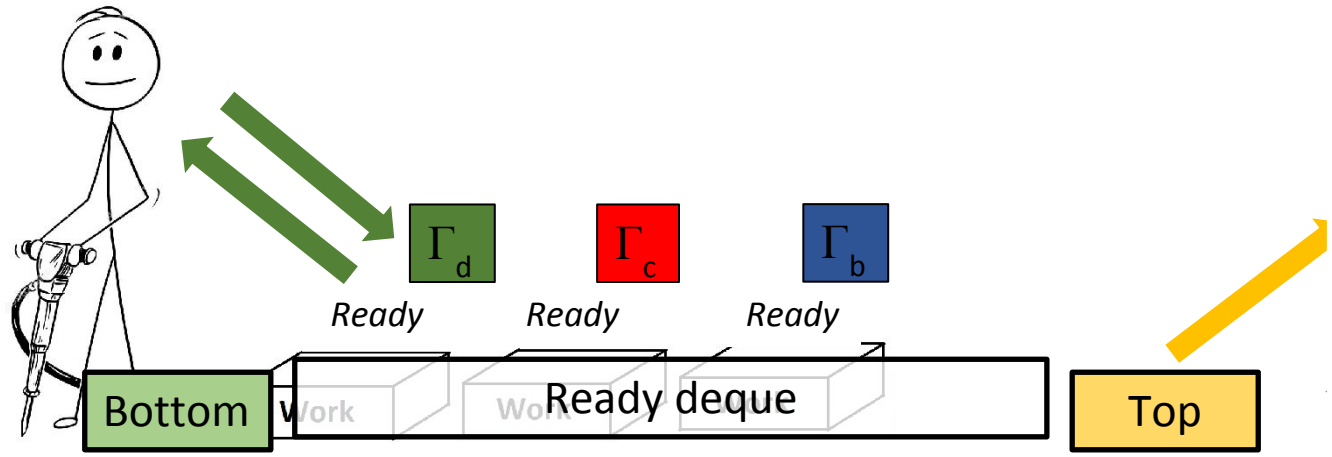
operación	C++	Java	Perl	Python	Ruby
Insertar elemento al final	`push_back`	`offerLast`	`push`	`append`	`push`
Insertar elemento al principio	`push_front`	`offerFirst`	`unshift`	`appendleft`	`unshift`
Eliminar el último elemento	`pop_back`	`pollLast`	`pop`	`pop`	`pop`
Eliminar el primer elemento	`pop_front`	`pollFirst`	`shift`	`popleft`	`shift`
Examinar el último elemento	`back`	`peekLast`	`$_[-1]`	`<obj>[-1]`	`last`
Examinar el primer elemento	`front`	`peekFirst`	`$_[0]`	`<obj>[0]`	`first`

Complejidad
En una implementación realizada con una lista doblemente enlazada la complejidad de todas las iteraciones es "**O(1)**", excepto para acceder a un elemento que no se encuentre en uno de los extremos de la estructura, donde la complejidad será "**O(n)**".

Si se implementa mediante un vector, la complejidad de las operaciones de la cola doblemente terminada coincide con la de la implementación con una lista.

Cola doblemente terminada de hilos en estado “ready”

- Esta deque tiene dos extremos:
  - Top (parte superior)
  - Bottom (parte inferior)
- Los hilos pueden ser insertados por la parte inferior (bottom) y pueden ser retirados por los dos extremos
- Un procesador trata esta estructura “ready deque” como una pila, colocando y retirando hilos de la parte inferior (bottom)
- Los hilos que son migrados a otros procesadores son retirados de la parte superior



Cola doblemente terminada de hilos en estado “ready”

- En general, un proceso obtiene trabajo sacando un hilo en estado ready de la parte inferior (bottom) de la deque.
- Entonces comienza a trabajar con el hilo, llamado Γ_a, y continua ejecutando instrucciones de Γ_a hasta que el hilo
  1. Genere otro hilo (spawn)
  2. Se postergue (stall)
  3. Muera (die)
  4. Habilite un hilo postergado (enable stalled thread)
- Para cada situación, en el algoritmo, se respetarán ciertas reglas

Reglas de ejecución de acuerdo con las cuatro situaciones del hilo

1. ***Spawn:*** Si el hilo Γ_a genera un hilo hijo Γ_b, el hilo padre Γ_a es colocado en la parte inferior (bottom) de la "ready deque" y el procesador comienza a trabajar el hilo Γ_b.
2. ***Stall:*** Si el hilo Γ_a se posterga, su procesador revisa la "ready deque". Si la "deque" tiene hilos, retira el hilo postergado y comienza a procesar el hilo que está primero en la parte inferior (bottom) de la "deque". Si la "deque" está vacía el procesador inicia el "work stealing": selecciona aleatoriamente otro procesador y roba el hilo que está en la parte superior (top) de la "deque" del procesador "víctima"
3. ***Die:*** Si el hilo Γ_a termina su ejecución, el procesador debe seguir la regla No. 2 para el hilo postergado (Stall)
4. ***Enable:*** Si la ejecución del hilo Γ_a habilita un hilo Γ_b que está postergado, el recién habilitado hilo Γ_b es colocado en la parte inferior (bottom) de la "deque" del procesador que estaba procesando el hilo Γ_a.

Reglas de ejecución de acuerdo con las cuatro situaciones del hilo

- En este algoritmo un hilo puede tener situaciones simultaneas:
  - Puede habilitar otro hilo (*Enable*) y detenerse (*Stall*)
    - Debe realizar primero la regla cuatro (Enable) y luego la regla dos (Stall)
  - Puede habilitar otro hilo (*Enable*) y morir (*Die*)
    - Debe realizar primero la regla cuatro (Enable) y luego la regla tres (Die)
- Estas reglas son similares al del algoritmo de hojas ocupadas, excepto la “nueva” regla cuatro.
  - Que es necesaria para que se mantengan propiedades estructurales, incluyendo la propiedad de hojas ocupadas

Algoritmo “Work-stealing”

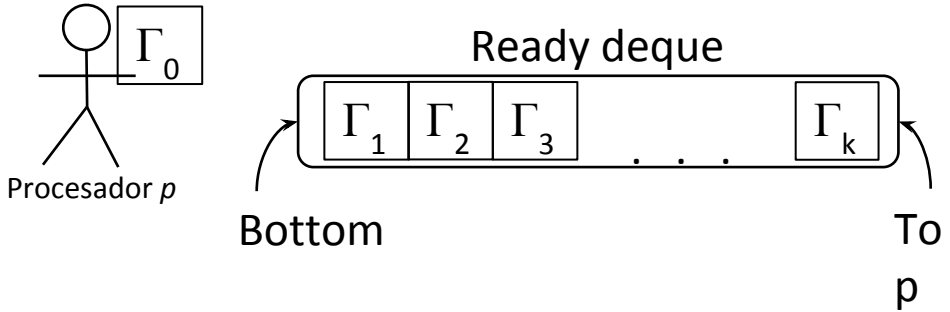
- El algoritmo Work-stealing comienza con todas las colas (ready deques) vacías.
- El hilo raíz (root thread) de la computación multihilo es colocado en la “ready deque” de uno de los procesadores, mientras los otros procesadores inician el “work-stealing”
- El “work-stealing” para un procesador tiene los siguientes pasos:
  - El procesador pasa al rol de ladrón (***thief***)
  - Intenta robar trabajo (work) de un procesador víctima (***victim***) seleccionado de forma aleatoria.
  - El ladrón consulta la “ready queue” de la víctima:
    - **Si la “ready deque” NO está vacía**, el ladrón saca el hilo de la parte superior (top) de la “ready deque” de la víctima y comienza a trabajar en él.
    - **Si la “ready deque” está vacía**, el ladrón selecciona otra víctima de forma aleatoria e intenta nuevamente.

Lema 4

- En la ejecución de alguna computación multihilos completamente estricta utilizando el algoritmo de *Work-Stealing*, considere algún procesador *p* y algún paso en el tiempo en el cual *p* esté procesando algún hilo. Sea Γ_0 el hilo que *p* está procesando, sean $\Gamma_1, \Gamma_2, \Gamma_3, \ldots \Gamma_k$ los hilos en la cola (*ready deque*) de *p* ordenados desde la parte inferior (*bottom*) hacia la parte superior (*top*) en la "ready deque", de tal forma que Γ_1 es el hilo que está más cerca de la parte inferior (bottom) y Γ_k el que está cerca de la parte superior (*top*) de la "ready deque". Si se tiene que $k > 0$, entonces los hilos en la "*ready deque*" del procesador *p* satisfacen las siguientes propiedades
  1. Para $i = 1,2,3, \ldots k$, el hilo Γ_i es el padre del hilo Γ_{i-1}
  2. Si $k > 1$, para $i = 1,2,3, \ldots k-1$, el hilo Γ_i no ha sido procesado desde que generó (creó) el hilo Γ_{i-1}

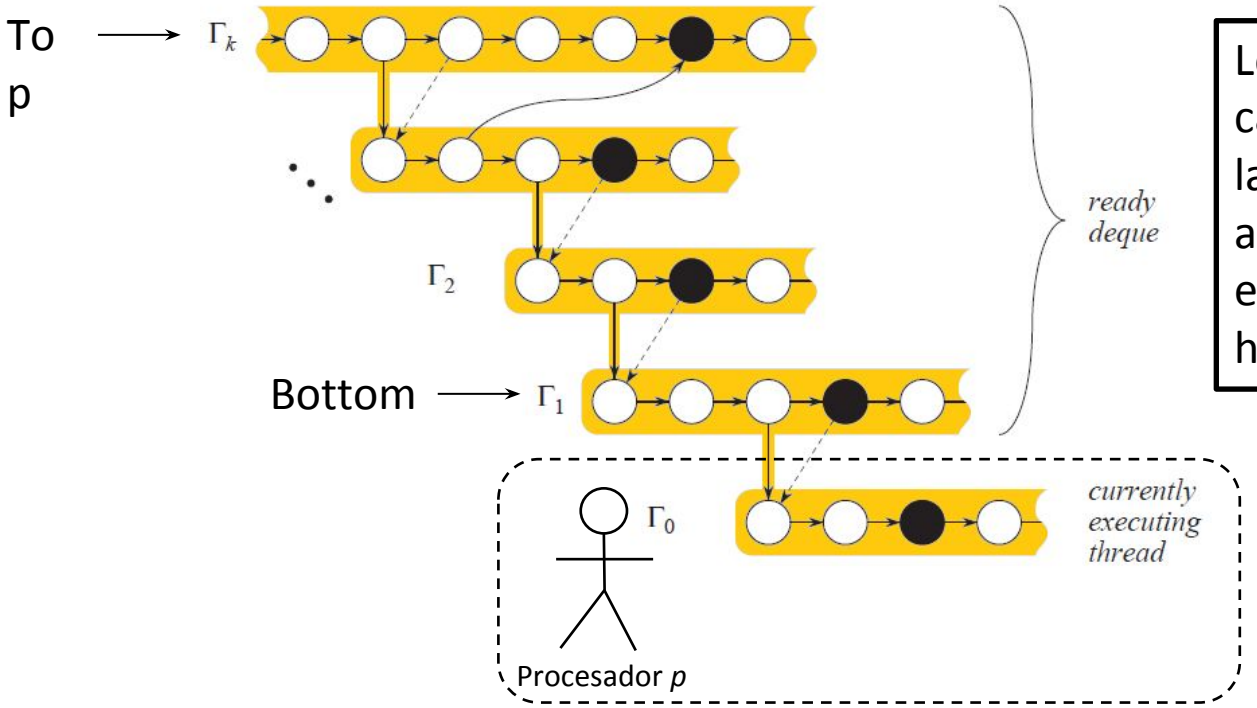
1. "Ser el hilo padre"

2. "Hilo padre no procesado"



Lema 4

Estructura de la “Ready deque” para un procesador



Los círculos negros en cada hilo representa la instrucción que actualmente está en estado ready para ese hilo.

Las flechas a trazos representan los “deque edges” que se presentan en la sección 6 del artículo.

Prueba lema 4

- La prueba es una inducción “sencilla” sobre el tiempo de ejecución.

1. La ejecución inicia con el hilo raíz en la “ready deque” de algún procesador y todas las otras “ready deque” vacías. De este modo, comenzando se cumple el lema cuatro.

Prueba lema 4

2. Ahora consideremos un **“paso”** del algoritmo en el cual el procesador p ejecuta una instrucción del hilo Γ_0. Sean Γ_1, Γ_2, Γ_3, … Γ_k los *k* hilos ubicados en la cola (*ready deque*) del procesador *p* ANTES de la ejecución del **“paso”** y vamos a suponer que
   - O *k* es igual a cero
   - O las dos propiedades se cumplen:
     - Γ_i es el padre del hilo Γ_{i-1}
     - Γ_i no ha sido procesado desde que creó Γ_{i-1}

1. “Ser el hilo padre”

2. “Hilo padre no procesado”

Prueba lema 4

- Sea Γ'_0, el hilo –si existe- que está siendo trabajado en el procesador p **después del "paso",** y sean Γ'_1, Γ'_2, Γ'_3, … Γ'_k los *k'* hilos en la "ready deque" del procesador *p* **después del "paso"**.
- Ahora, usando las reglas del algoritmo se debe mostrar que se preservan las condiciones del lema.
  - O *k'* es igual a cero
  - O las dos propiedades se cumplen:
    - Γ'_i es el padre del hilo Γ'_{i-1}
    - Γ'_i no ha sido procesado desde que creó Γ'_{i-1}

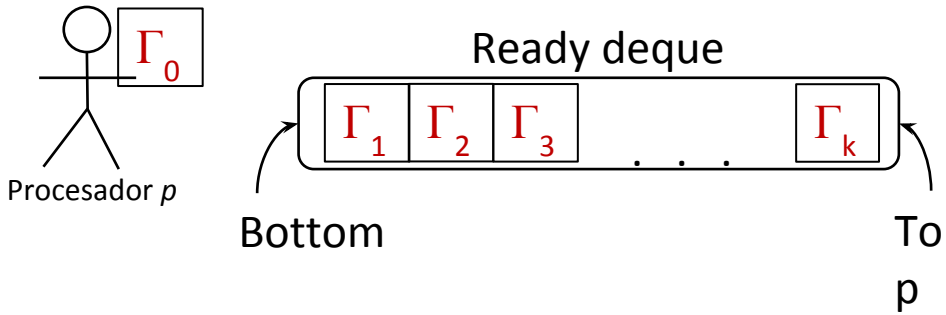
1. "Ser el hilo padre"

2. "Hilo padre no procesado"

Prueba lema 4

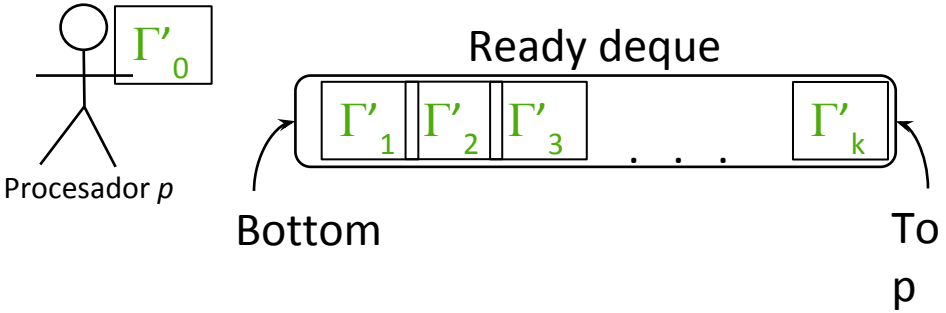
Antes del "Paso" **se cumplen** las condiciones del lema.

Antes del "Paso" en el que el procesador *p* ejecuta instrucciones del hilo Γ_0



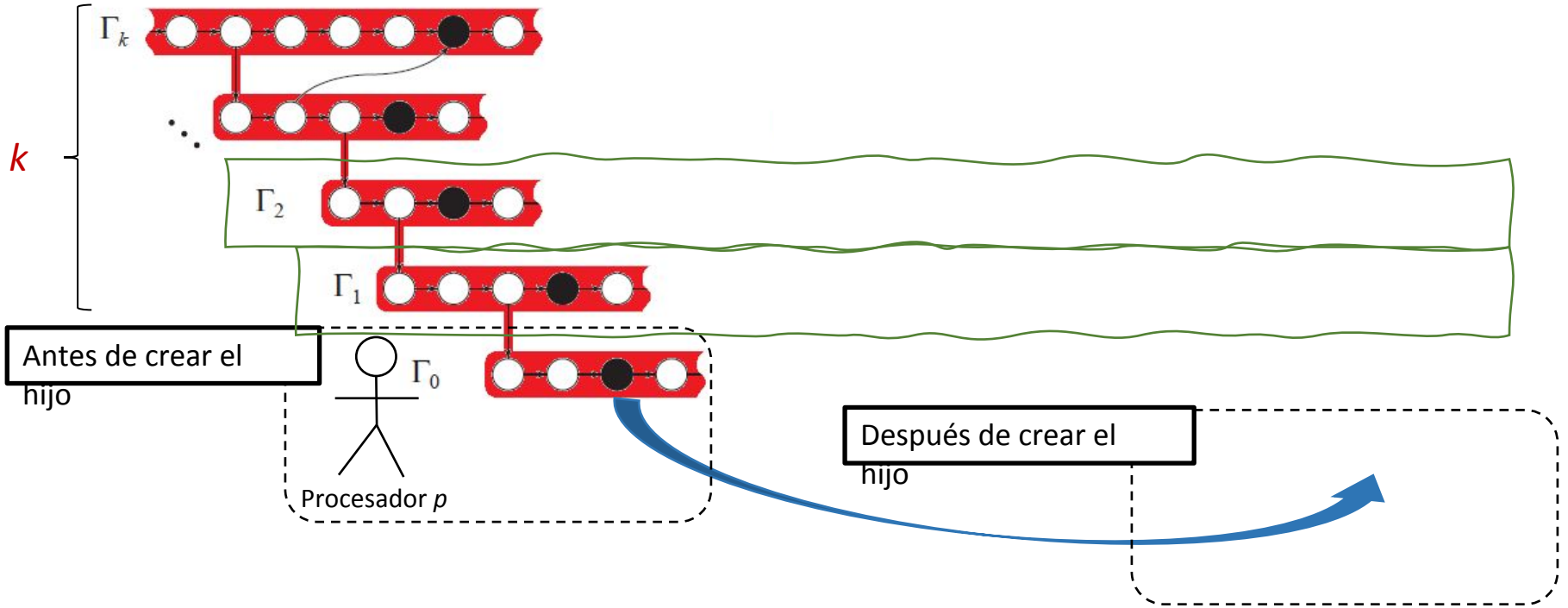
Usando las reglas del algoritmo, se debe comprobar que después del "Paso" se preservan las condiciones del lema.

Después del "Paso" el procesador *p* ejecuta instrucciones del hilo Γ'_0



Prueba lema 4

- ***Regla 1, Spawn.*** Si Γ_0 crea un hijo, entonces p coloca (push) Γ_0 en la parte inferior (bottom) de la "ready deque" y comienza a trabajar en el hijo. Así, Γ'_0 es el hijo, y se tendrá que $k' = k + 1 > 0$ y para j = 1, 2, 3, …, k', se tiene que $\Gamma'_i = \Gamma_{i-1}$.



Prueba lema 4

- Ahora debemos verificar las dos propiedades para la ***Regla 1, Spawn.***
  - **Propiedad 1:** Si $k' > 1$, entonces para j = 2, 3, 4, …, k' el hilo Γ'_j es el padre de Γ'_{j-1}, debido a que antes de la creación (spawn) del hijo se sabía que para $k > 0$ para i = 1, 2, 3, …, k, el hilo Γ_i es el padre de Γ_{i-1}. Además Γ'_1 [que antes de tener hijos se llamaba Γ_0] es el padre de Γ'_0.
  - **Propiedad 2:** Si $k' > 2$, entonces para j = 2, 3, 4, …, $k'-1$ el hilo Γ'_j no ha sido trabajado ya que él creó Γ'_{j-1}, esto porque antes de crear (spawn) el hijo se sabía que para $k > 1$, con i = 1, 2, 3, …, $k - 1$, el hilo Γ_i no había sido trabajado desde que creó a Γ_{i-1}. Finalmente, el hilo Γ'_1 [que antes de tener hijos se llamaba Γ_0] no ha sido trabajado desde que creó a Γ'_0, pues la creación (spawn) recien había ocurrido.

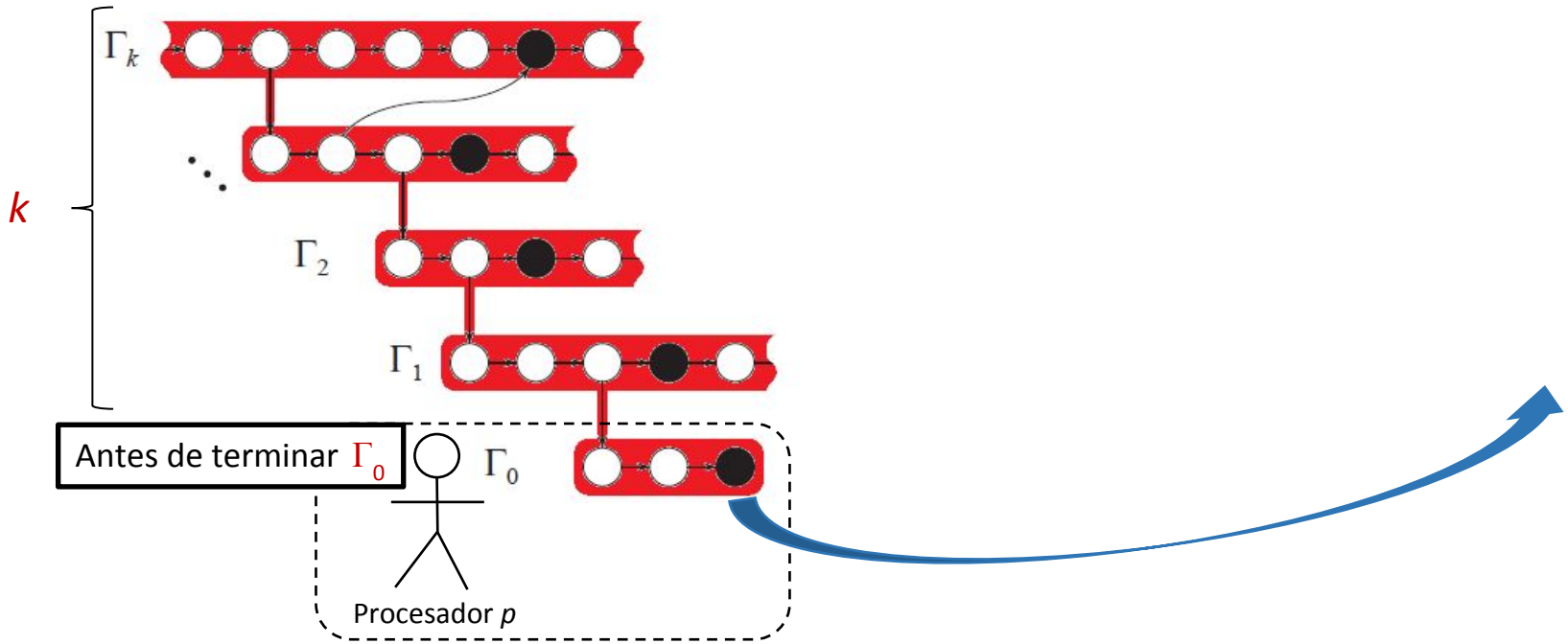
1. “Ser el hilo padre”

2. “Hilo padre no procesado”

Prueba lema 4

- ***Regla 2, Stall y Regla 3, Die.*** Si el hilo Γ_0 se posterga o termina, hay dos casos para considerar.
  - Si $k = 0$, la "ready deque" estará vacía y el procesador iniciará el "work stealing" y cuando el procesador consiga (robe) un hilo y comience a trabajarlo, se tendrá que $k' = 0$.
  - Si $k > 0$, la "ready deque" NO estará vacía, así que el procesador sacará (pop) el hilo que está primero en la parte inferior de la "ready deque" y comenzará a trabajarlo. De este modo, $\Gamma'_0 = \Gamma_1$ (el hilo que fue sacado de la "ready deque") y $k' = k - 1$ y para j = 1, 2, 3, …, k' se tiene que $\Gamma'_j = \Gamma_{j+1}$.

Prueba lema 4



Prueba lema 4

- Ahora, si $k' > 0$, se pueden revisar las dos propiedades para la ***Regla 2, Stall*** *y la* ***Regla 3, Die***.
  - **Propiedad 1:** Para j = 1, 2, 3, …, k', el hilo Γ'_j es el padre del hilo Γ'_{j-1}, debido a que para i = 1, 2, 3, …, k el hilo Γ_i es el padre de Γ_{i-1}. 1. "Ser el hilo padre"
  - **Propiedad 2:** Si $k' > 1$, entonces para j = 1, 2, 3, …, $k'-1$ el hilo Γ'_j no ha sido trabajado debido a que él creó Γ'_{j-1}, esto porque antes de postergarse (stall) o de terminar (die) se sabía que para $k > 2$, con i = 2, 3, 4, …, $k - 1$, el hilo Γ_i no había sido trabajado desde que creó a Γ_{i-1}. 2. "Hilo padre no procesado"

Prueba del lema 4

- ***Regla 4, Enable.*** Si la ejecución de Γ_0 habilita un hilo que está postergado (Stall) entonces, debido a la condición de planificador "completamente estricto", el hilo previamente postergado debe ser el padre de Γ_0.
- Debemos observar que *k* debe ser 0 (*k* = 0).
  - Si *k* > 0, la "ready queue" del procesador NO estaría vacía y su hilo padre debe ser el hilo que está de primero en el extremo inferior (bottom) de la "ready deque". En este caso el hilo padre estaría en estado "ready" y NO aplica la regla 4.

Prueba del lema 4

- Con k = 0, la “ready deque” estará vacía y el procesador colocará al hilo padre en el extremo inferior (bottom) de la ready deque.
  - En este caso $\Gamma'_0 = \Gamma_0$ y $k' = k + 1$ con Γ'_1 representando el padre recién habilitado.
- En este caso solo es necesario validar la primera propiedad.
- **Propiedad 1:** El hilo Γ'_1 obviamente es el padre del hilo Γ'_0.

Prueba del lema 4

- Si algún otro procesador roba un hilo del procesador p, se debe tener que $k > 0$, y después del robo se tendrá que $k' = k - 1$. Si $k' > 0$ se mantiene, entonces "claramente" las dos propiedades de mantienen
- Todas las otras acciones del procesador p –tales como work stealing, o ejecutar una instrucción que no invoca alguna de las reglas anteriores- "claramente" preservan el lema 4.

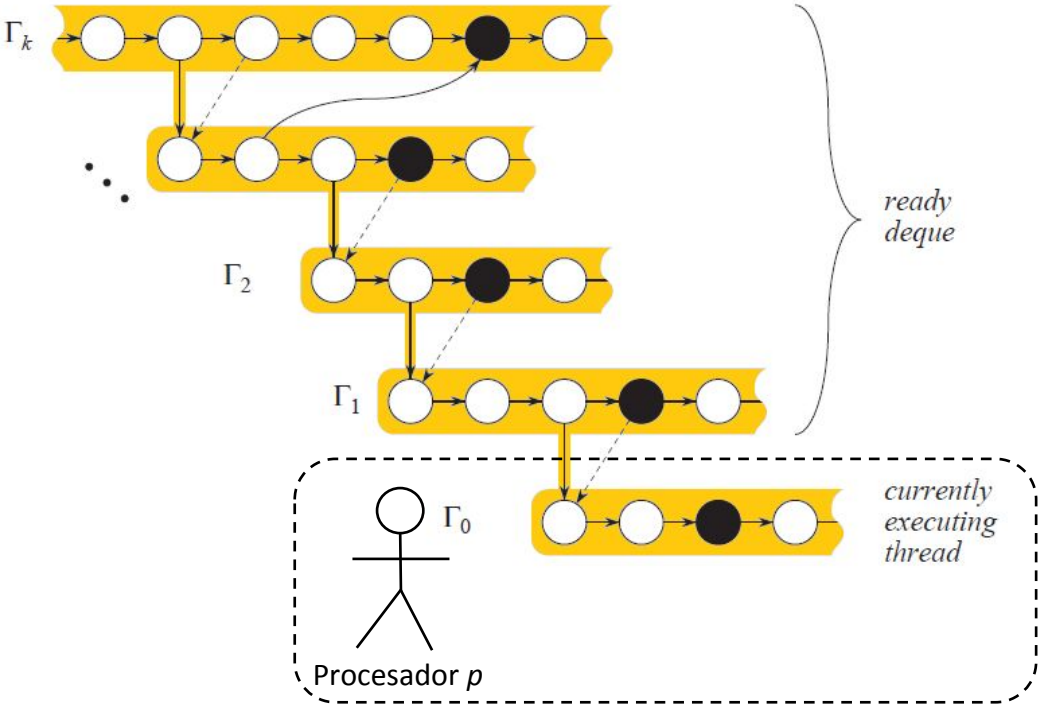
Γ_k pudo ser ejecutado después que creó el último hijo

- ¿Cómo puede ocurrir que el hilo Γ_k hubiese sido trabajado desde que él creó Γ_{k-1} ya que la propiedad 2 excluye a Γ_k?
- Esta situación ocurre cuando Γ_k es robado de la "ready deque" del procesador p y entonces se posterga (stall) en su nuevo procesador.
- Posteriormente Γ_k es habilitado nuevamente por Γ_{k-1} y llevando de regreso a la "ready deque" del procesador p.
- La observación clave es que cuando Γ_k es habilitado nuevamente, la "ready deque" del procesador p está vacía y p está trabajando en Γ_{k-1}
- Los otros hilos, Γ_{k-2}, Γ_{k-3},, Γ_{k-4}, …, Γ_0, mostrados en la siguiente figura fueron creados después que Γ_k fuera habilitado nuevamente.

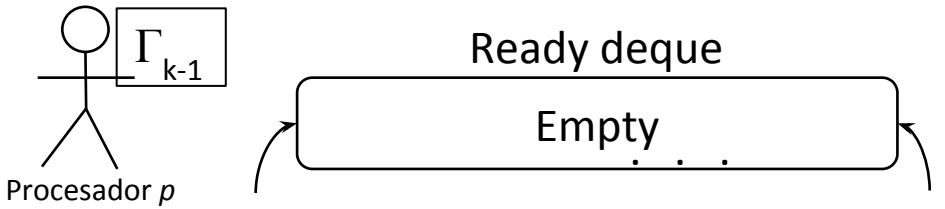
Γ_k pudo ser ejecutado después que creó el último hijo

Γ_k pudo ser ejecutado después que creó el último hijo y estar en la ready deque de un procesador.

Estructura de la “Ready deque” para un procesador



Γ_k pudo ser ejecutado después que creó el último hijo



La observación clave es que cuando Γ_k es habilitado nuevamente, la "ready deque" del procesador p está vacía y p está trabajando en Γ_{k-1}

p

Teorema 5

- Para alguna computación multihilo completamente estricta con un tamaño de pila (stack depth) S_1, el algoritmo Work Stealing que se ejecuta sobre un computador con P procesadores utiliza un espacio de S_1P
  - De manera similar al algoritmo de hojas ocupadas, el algoritmo de work stealing garantiza la propiedad de hojas ocupadas: en cualquier paso de la ejecución, cada hilo hoja en el subárbol de generación actual tiene un procesador trabajando en él.
  - Si se logra establecer este hecho, la prueba se completa con el lema 2.

Prueba Teorema 5

- Que el algoritmo de Work stealing garantiza la propiedad de hojas ocupadas es una consecuencia “simple” del lema 4.
  - En cada paso, una hoja en el subárbol de generación actual debe estar en estado ready y por tanto debería tener:
    - O un procesador trabajando en él.
    - O estar en la “ready deque” de un procesador.
  - Sin embargo, el lema 4 garantiza que ningún hilo hoja vaya a la “ready deque” de un procesador mientras el procesador esté ejecutando algún otro hilo.
- Ver lema 2… ☺

¿Qué sigue?

- Con el acotamiento en el espacio, ahora podemos pasar a atender el análisis de los límites en el tiempo y en las comunicaciones para el algoritmo de Work Stealing.
- Sin embargo, antes debemos definir un modelo para gestionar la contención que puede resultar en el momento que múltiples procesadores “ladrones” intenten robar de manera simultanea a la misma víctima.

Gracias